

Natural Language Processing

Comprehensive Question & Answer Guide

Covering 17 Exam Questions with Detailed Explanations,
Diagrams, Trees, Tables & Examples

*Topics: Tokenization • Morphology • POS Tagging • CFG • PCFG
Parsing • Semantics • HMM • FST • Stemming & Lemmatization*

Q1. Part-of-Speech (POS) Tagging in NLP

Definition

Part-of-Speech (POS) Tagging is the process of assigning a grammatical label (or tag) to each word in a sentence based on its definition and its context in the sentence. These labels (tags) indicate whether a word is a Noun, Verb, Adjective, Adverb, Pronoun, Preposition, Conjunction, Interjection, Determiner, etc.

Example of POS Tagging

Sentence: "The cat sat on the mat"

Word	POS Tag	Tag Meaning
The	DT	Determiner
cat	NN	Noun (singular)
sat	VBD	Verb (past tense)
on	IN	Preposition
the	DT	Determiner
mat	NN	Noun (singular)

Common POS Tags (Penn Treebank Tagset)

Tag	Meaning	Example
NN	Noun, singular	dog, city
NNS	Noun, plural	dogs, cities
VB	Verb, base form	run, eat
VBD	Verb, past tense	ran, ate
JJ	Adjective	big, fast
RB	Adverb	quickly, very
PRP	Personal pronoun	I, he, she
DT	Determiner	the, a, an
IN	Preposition	in, on, at
CC	Coordinating conjunction	and, but, or

Approaches to POS Tagging

1. Rule-Based Tagging

Uses hand-written linguistic rules. Example: If a word ends in '-ing' and follows a verb, tag it as VBG (gerund/present participle). Systems like ENGTWOL use morphological analysis and context rules.

2. Stochastic/Probabilistic Tagging (HMM-based)

Uses Hidden Markov Models (HMM). It computes the most probable tag sequence using transition probabilities ($P(\text{tag}_i | \text{tag}_{\{i-1\}})$) and emission probabilities ($P(\text{word} | \text{tag})$).

$$\text{Tag} = \operatorname{argmax} P(\text{word} | \text{tag}) \times P(\text{tag} | \text{previous_tag})$$

3. Transformation-Based Tagging (Brill Tagger)

Starts with initial tags, then applies learned transformation rules to fix errors. E.g., 'Change NN to VB if the previous word is TO.'

4. Deep Learning / Neural Tagging

Uses LSTM, BiLSTM, BERT, or Transformer-based models. These models learn contextual embeddings and achieve state-of-the-art results (>97% accuracy on standard datasets).

Need / Importance of POS Tagging in NLP

- Word Sense Disambiguation: The word 'bank' can be a noun (river bank) or a verb (to bank money). POS helps disambiguate.
- Named Entity Recognition (NER): Identifying names, places, and organizations relies on POS tags.
- Parsing and Syntactic Analysis: Grammatical structure analysis depends on POS tags.
- Information Extraction: Extracting facts from text requires understanding grammatical roles.
- Machine Translation: Correct translation requires knowing the part of speech of each word.
- Text-to-Speech: Pronunciation varies by POS (e.g., 'record' as noun vs. verb).
- Question Answering, Summarization, Sentiment Analysis: All rely heavily on POS information.

Q2. Natural Languages vs. Programming Languages

Introduction

Natural Languages (NL) are languages that evolved naturally through human use over centuries, such as English, Hindi, French, etc. Programming Languages (PL) are formal languages designed by humans to instruct computers, such as Python, Java, C++.

Feature	Natural Language	Programming Language
Origin	Evolved naturally over centuries	Designed and formally specified by humans
Ambiguity	Highly ambiguous (lexical, structural, pragmatic)	Completely unambiguous; each construct has one meaning
Grammar	Descriptive, flexible, has exceptions	Prescriptive, rigid, strictly defined by BNF/EBNF

Vocabulary	Unlimited, dynamic, context-dependent	Fixed set of keywords, operators, and identifiers
Redundancy	High redundancy; context resolves ambiguity	No redundancy; every token is significant
Context Dependence	Heavily context-dependent (pragmatics, world knowledge)	Context-free or context-sensitive, formally defined
Creativity	Allows metaphor, humor, sarcasm, irony	No metaphor; strict logical meaning only
Evolution	Changes over time (new words, usage)	Changes only via versioning by designers
Interpretation	Requires human intelligence to fully understand	Interpreted/compiled deterministically by machines
Error Tolerance	Understood even with grammatical errors	Syntax errors halt compilation/execution
Examples	English, Hindi, French, Japanese	Python, Java, C, JavaScript, SQL

Why Natural Language is Hard for Computers

- Ambiguity: 'I saw the man with a telescope.' – Did I use the telescope, or did the man have it?
- Anaphora Resolution: 'John told Peter that he was late.' – Who is 'he'?
- Idioms: 'It's raining cats and dogs' does not literally mean animals are falling.
- Sarcasm and Irony: 'Oh great, another Monday!' means the opposite of what is literally said.
- Unknown Words: New slang, technical jargon, brand names.

Structure Comparison

Programming language syntax is defined formally. For example, Python's if-statement:

```
if condition:
    statement
```

Natural language 'if' can take many forms: 'If it rains, I'll stay home,' 'Should it rain, I'll stay home,' 'I'll stay home if it rains,' etc. — all equivalent in meaning but structurally different.

Q3. Tokenization in NLP

Definition

Tokenization is the process of splitting a raw text string into a list of tokens — meaningful units such as words, subwords, characters, or sentences. It is one of the very first and most fundamental steps in any NLP pipeline.

Input: "Natural Language Processing is fascinating!"

Tokens: ["Natural", "Language", "Processing", "is", "fascinating", "!"]

Why Tokenization is Important

- Converts unstructured text into a structured form that can be processed by algorithms.
- Helps identify word boundaries, sentence boundaries, and meaningful units.
- Enables downstream tasks: POS tagging, parsing, NER, sentiment analysis, etc.

Types of Tokenization

1. Word Tokenization

Splits text by whitespace and punctuation into individual words. Most common form.

```
Input : "Don't stop believing!"  
Output: ["Don't", "stop", "believing", "!"]
```

Challenge: Contractions ('don't' → 'do' + 'n't?'), hyphenated words (state-of-the-art), abbreviations (U.S.A.).

2. Sentence Tokenization (Sentence Segmentation)

Splits a document into individual sentences. Used before word tokenization in many pipelines.

```
Input : "Dr. Smith went to the U.S.A. He loved it."  
Output: ["Dr. Smith went to the U.S.A.", "He loved it."]
```

Challenge: Periods in abbreviations (Dr., Mr., U.S.A.) vs. sentence-ending periods.

3. Subword Tokenization

Splits words into smaller meaningful units (subwords or byte-pairs). Used in modern deep learning NLP models like BERT, GPT, etc.

Algorithms: Byte-Pair Encoding (BPE), WordPiece, SentencePiece, Unigram.

```
Input : "unbelievable"  
BPE   : ["un", "##believ", "##able"] (WordPiece notation)
```

Advantage: Handles rare and out-of-vocabulary words by decomposing them into known subword units.

4. Character Tokenization

Splits text into individual characters. Useful for languages without clear word boundaries (Chinese, Japanese) or for character-level language models.

```
Input : "Hello"  
Output: ['H', 'e', 'l', 'l', 'o']
```

5. N-gram Tokenization

Creates tokens of N consecutive words (or characters). Useful for capturing local context.

```
Input : "I love NLP"  
Bigrams (2-gram): ["I love", "love NLP"]  
Trigrams (3-gram): ["I love NLP"]
```

6. Tweet / Social Media Tokenization

Specialized tokenizers handle hashtags (#NLP), mentions (@user), emoticons (:)), and URLs.
Example: NLTK TweetTokenizer.

Challenges in Tokenization

- Contractions: 'can't' → 'can' + 'not' or keep as one token?
- Multi-word expressions: 'New York', 'ice cream' — one token or two?
- Languages without spaces: Chinese and Japanese use characters not spaces.
- Compound words: German 'Lebensversicherungsgesellschaft' (life insurance company).

Q4. Natural Language Processing (NLP) – Definition & Stages

Definition

Natural Language Processing (NLP) is a subfield of Artificial Intelligence (AI) and Computational Linguistics that focuses on enabling computers to understand, interpret, generate, and interact with human (natural) language in a meaningful and useful way.

NLP bridges the gap between human communication and computer understanding. It combines techniques from linguistics, computer science, and machine learning to process and analyze large amounts of natural language data.

Applications of NLP

- Machine Translation (Google Translate)
- Sentiment Analysis (Product reviews, social media)
- Chatbots and Virtual Assistants (Siri, Alexa, ChatGPT)
- Information Extraction (Named Entity Recognition)
- Speech Recognition (voice-to-text)
- Text Summarization and Question Answering

Stages / Levels of NLP Processing

NLP processing is organized into a pipeline of stages, each building on the previous one:

NLP Processing Pipeline (Top to Bottom)	
① Morphological & Lexical Analysis	→ Tokenization, POS Tagging, Stemming/Lemmatization
② Syntactic Analysis (Parsing)	→ Parse Trees, Dependency Trees, CFG/PCFG
③ Semantic Analysis	→ Word Sense Disambiguation, Semantic Role Labeling
④ Discourse Integration	→ Coreference Resolution, Discourse Structure
⑤ Pragmatic Analysis	→ Speech Acts, Intention, Context Understanding

Stage 1: Morphological & Lexical Analysis

The first stage deals with individual words and their internal structure.

- Tokenization: Breaking text into words, sentences, or subwords.

- POS Tagging: Labeling each word with its grammatical category.
- Stemming & Lemmatization: Reducing words to their base/root form.
- Stop-word Removal: Removing common, low-information words (the, is, at).
- Morphological Analysis: Analyzing word structure (prefixes, suffixes, roots).

Example: "running" → lemma: "run", POS: VBG

Stage 2: Syntactic Analysis (Parsing)

Analyzes the grammatical structure of sentences. Produces parse trees that show relationships between words.

- Context-Free Grammar (CFG) parsing: CYK, Earley, Recursive Descent algorithms.
- Dependency Parsing: Establishes dependency relations between words.
- Constituency Parsing: Groups words into nested phrases (NP, VP, PP).

Example: $S \rightarrow NP VP$ | $NP \rightarrow Det Noun$ | $VP \rightarrow Verb NP$

Stage 3: Semantic Analysis

Determines the meaning of sentences. Identifies what the sentence actually means.

- Word Sense Disambiguation (WSD): Choosing the correct meaning of ambiguous words.
- Semantic Role Labeling (SRL): Identifies agent, patient, instrument, etc.
- Named Entity Recognition (NER): Identifies persons, places, organizations, dates.
- Semantic Parsing: Converts natural language to formal logical representations.

Example: "John ate an apple." → Agent(eat, John), Patient(eat, apple)

Stage 4: Discourse Integration

Analyzes meaning across multiple sentences in a text, not just within individual sentences.

- Coreference Resolution: Identifying that 'he' and 'John' refer to the same entity.
- Discourse Structure: Understanding how sentences relate (contrast, cause, sequence).
- Anaphora Resolution: Resolving pronouns and definite noun phrases.

Example: "John won the race. He trained hard." → 'He' = 'John'

Stage 5: Pragmatic Analysis

The highest level of analysis. Determines the intended meaning considering real-world context and communicative intent.

- Speech Act Theory: Understanding whether a statement is a request, promise, question, etc.
- Presupposition: What the speaker takes for granted.
- Implicature: What is implied but not directly stated.
- Grounding: Mutual understanding between participants in dialogue.

Example: "Can you pass the salt?" is a request, not a yes/no question about ability.

Q5. Challenges of Natural Language Processing

Overview

Despite tremendous progress, NLP faces numerous fundamental challenges rooted in the complexity of human language. Below are the major challenges:

1. Ambiguity

Ambiguity is the single biggest challenge in NLP. It occurs at multiple levels:

- Lexical Ambiguity: A word has multiple meanings. 'Bank' = financial institution OR river bank.
- Syntactic/Structural Ambiguity: A sentence can be parsed in multiple ways. 'I saw the man with a telescope' – who had the telescope?
- Semantic Ambiguity: 'Visiting relatives can be boring' – relatives are visiting, OR you are visiting them?
- Pragmatic Ambiguity: 'Can you open the window?' – a question about ability, or a request?

2. Variability and Synonymy

The same concept can be expressed in countless ways. 'The vehicle was speeding,' 'The car was going too fast,' 'The automobile exceeded the speed limit' — all equivalent, but textually different.

3. Context Dependency

Understanding language requires extensive world knowledge and context. 'He poured water on the fire.' vs. 'He poured oil on the fire.' — similar structure, vastly different implications.

4. Co-reference and Anaphora Resolution

Pronouns and definite NPs refer to previously mentioned entities: 'Maria told Lisa that she would win.' — who is 'she'? This requires reasoning about context and entities.

5. Named Entity Recognition (NER) Challenges

'Paris' can be a city, a person's name, or a brand. 'Apple' can be a fruit or a technology company. Correctly identifying named entities in context is difficult.

6. Idioms, Sarcasm, and Figurative Language

Idioms cannot be understood literally: 'kick the bucket' = die. Sarcasm: 'Oh brilliant, the server crashed again!' — negative sentiment despite positive word 'brilliant'.

7. Low-Resource Languages

Most NLP research and data is in English. Thousands of languages have little or no annotated data, making it hard to build NLP systems for them.

8. Morphological Complexity

Languages like Turkish, Finnish, and Arabic have extremely rich morphology. A single word can encode what requires an entire English sentence.

'Çekoslovakyalılaştıramadıklarımızdanmışsınızcasına' is a valid single Turkish word.

9. Handling Noisy Text

Social media text contains typos, abbreviations, hashtags, emoticons, and non-standard grammar.

'omg this is sooo gd lol #awesome :D' is difficult for standard NLP tools.

10. Computational Cost and Scalability

Modern deep learning NLP models (GPT-4, BERT) have billions of parameters and require enormous computational resources (GPUs/TPUs), making them inaccessible for many applications.

Challenge	Example	Impact
Ambiguity	Bank (river/financial)	Wrong interpretation
Coreference	He/she refers to?	Wrong entity linking
Sarcasm	'Great job!' (negative)	Wrong sentiment
OOV Words	New slang terms	Unknown tokens
Low-resource	Rare languages	No training data

Q6. Top-Down Depth-First Left-to-Right Parse Tree

Grammar Rules (Given)

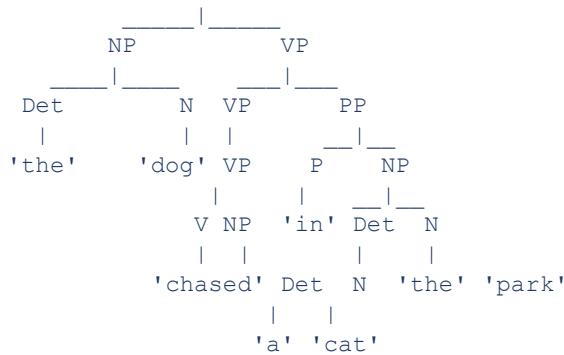
Consider the following Context-Free Grammar (CFG):

S → NP VP
NP → Det N | Det N PP | PRP
VP → V NP | VP PP
PP → P NP
Det → 'the' | 'a'
N → 'dog' | 'cat' | 'park' | 'man'
V → 'saw' | 'chased'
P → 'in' | 'with'
PRP → 'he' | 'she'

Sentence to Parse: "The dog chased a cat in the park"

Parse Tree (ASCII Representation)

S



Top-Down Depth-First Parse Process

Top-down parsing starts from the Start Symbol (S) and tries to match the input by expanding rules. Depth-first means we explore one branch fully before backtracking to try alternatives. Left-to-right means we match the leftmost word first.

Step	Stack (Top → Bottom)	Input Remaining	Action
1	S	the dog chased a cat in the park	Expand S → NP VP
2	NP, VP	the dog chased a cat in the park	Expand NP → Det N
3	Det, N, VP	the dog chased a cat in the park	Match Det → 'the'
4	N, VP	dog chased a cat in the park	Match N → 'dog'
5	VP	chased a cat in the park	Expand VP → VP PP
6	VP, PP	chased a cat in the park	Expand VP → V NP
7	V, NP, PP	chased a cat in the park	Match V → 'chased'
8	NP, PP	a cat in the park	Expand NP → Det N
9	Det, N, PP	a cat in the park	Match Det → 'a'
10	N, PP	cat in the park	Match N → 'cat'
11	PP	in the park	Expand PP → P NP
12	P, NP	in the park	Match P → 'in'
13	NP	the park	Expand NP → Det N
14	Det, N	the park	Match Det → 'the'
15	N	park	Match N → 'park'
16	—	—	SUCCESS: Parse complete!

Key Characteristics of Top-Down Parsing

- Starts from the Start Symbol (S) and works down to terminal symbols (words).
- Depth-First: Explores one branch completely before trying alternatives.
- Left-to-Right: Processes input tokens from left to right.
- May require backtracking when a chosen rule doesn't match the input.
- Recursive Descent Parser is the most common implementation.

Q7. Derivational and Inflectional Morphology

What is Morphology?

Morphology is the branch of linguistics that studies the internal structure of words — how words are formed from smaller meaningful units called morphemes. A morpheme is the smallest unit of meaning in a language.

'unhappiness' = un + happy + ness (3 morphemes)

Types of Morphemes

- Free Morphemes: Can stand alone as independent words. E.g., 'book', 'run', 'happy'.
- Bound Morphemes: Cannot stand alone; must attach to another morpheme. E.g., '-ing', 'un-', '-ness', '-ed'.

1. Inflectional Morphology

Inflectional morphology refers to the modification of a word to express different grammatical categories such as tense, number, gender, case, or person — WITHOUT changing the word's basic category (POS) or core meaning. Inflection is a grammatical process.

Inflectional Feature	Morpheme	Example	Meaning Change
Plural (number)	-s / -es	cat → cats	One to many
Past tense	-ed	walk → walked	Present to past
Present participle	-ing	run → running	Ongoing action
Comparative	-er	fast → faster	More of quality
Superlative	-est	fast → fastest	Most of quality
Possessive	's	John → John's	Ownership
3rd person singular	-s	run → runs	Subject agreement

Key Property of Inflection: The POS does NOT change. 'walk' (verb) → 'walked' (still a verb). 'cat' (noun) → 'cats' (still a noun). The word stays in the same lexical class.

2. Derivational Morphology

Derivational morphology refers to the formation of a new word with a new meaning (and often a new POS) by adding a derivational affix (prefix or suffix) to a base word. It 'derives' a new word.

Base Word (POS)	Affix	Derived Word (POS)	Change
happy (Adj)	-ness	happiness (Noun)	Adj → Noun
teach (Verb)	-er	teacher (Noun)	Verb → Noun
nation (Noun)	-al	national (Adj)	Noun → Adj
quick (Adj)	-ly	quickly (Adverb)	Adj → Adverb
predict (Verb)	un-	unpredictable (Adj)	Verb → Adj
do (Verb)	un-	undo (Verb)	New meaning, same POS
friend (Noun)	-ship	friendship (Noun)	New meaning, same POS
active (Adj)	-ate	activate (Verb)	Adj → Verb

Key Property of Derivation: The POS or meaning often CHANGES. 'teach' (verb) → 'teacher' (noun). A new lexical entry is created.

Comparison: Inflectional vs. Derivational

Feature	Inflectional	Derivational
POS change?	NO (always same POS)	YES (usually changes POS)
Meaning change?	Minor (grammatical only)	YES (creates new word)
Productivity	Highly regular and productive	Less regular, more exceptions
Position	Usually outermost suffix	Closer to root than inflectional
Example	cat → cats (still noun)	happy → happiness (noun)
Dictionary entry?	No new entry	Creates new dictionary entry

Q8. Relationships Between Word Meanings

Introduction

The study of meaning in language is called Semantics. Words can relate to each other in various ways based on their meanings. These semantic relationships are fundamental to NLP tasks like Word Sense Disambiguation, Information Retrieval, Ontology Building, and Knowledge Graphs (e.g., WordNet).

1. Homonymy

Homonyms are words that share the same spelling and/or pronunciation but have completely unrelated, different meanings. The separate meanings historically come from entirely different words.

- Bank: (i) a financial institution (ii) the sloping land beside a river
- Bark: (i) the sound a dog makes (ii) the outer covering of a tree (iii) a type of ship
- Bat: (i) a flying mammal (ii) a cricket/baseball bat (iii) to bat an eyelid
- Fly: (i) to travel through air (ii) an insect (iii) the flap of a tent

Types of Homonymy:

- Homophones: Same pronunciation, different spelling: 'write' / 'right', 'flour' / 'flower'
- Homographs: Same spelling, different pronunciation and meaning: 'lead' (to lead) / 'lead' (metal)

2. Polysemy

A polysemous word has multiple related meanings that evolved from the same origin. The meanings are connected through extension of use.

- Head: (i) top of body (ii) head of an organization (iii) head of a river (iv) head of a bed
- Bright: (i) giving off light (ii) intelligent (iii) cheerful
- Run: (i) to run fast (ii) run a company (iii) a run of bad luck (iv) a run in a stocking

Polysemy vs. Homonymy: In polysemy, the meanings are RELATED (same origin). In homonymy, the meanings are UNRELATED (different origins). 'Bank' (financial/river) = Homonymy. 'Head' (body part/leader) = Polysemy.

3. Synonymy

Synonyms are words with the same or very similar meaning. True synonymy (identical meaning in all contexts) is rare; most synonyms differ in connotation, register, or usage.

- Begin / Start / Commence / Initiate
- Happy / Glad / Joyful / Elated / Content
- Big / Large / Huge / Enormous / Gigantic
- Fast / Quick / Rapid / Swift

Note: 'Start' and 'Commence' are synonyms, but 'commence' is more formal. True semantic equivalence rarely holds in all contexts.

4. Antonymy

Antonyms are words with opposite meanings.

- Complementary (binary opposites): alive/dead, true/false, married/unmarried
- Gradable antonyms (on a scale): hot/cold, tall/short, fast/slow
- Relational antonyms: buy/sell, teacher/student, parent/child, employer/employee

5. Hyponymy (Is-A Relationship)

Hyponymy is the semantic relationship where the meaning of one word (hyponym) is included in the meaning of another (hypernym). It forms hierarchical IS-A relationships — the foundation of taxonomic ontologies.

```
Animal (hypernym)
└─ Mammal (hyponym of Animal, hypernym of Dog)
    └─ Dog (hyponym of Mammal)
        └─ Poodle (hyponym of Dog)
```

- 'Rose' is a hyponym of 'Flower'. 'Flower' is the hypernym of 'Rose'.
- 'Toyota' is a hyponym of 'Car'. 'Car' is a hyponym of 'Vehicle'.
- Co-hyponyms: Words that share the same hypernym: 'Rose', 'Lily', 'Tulip' are co-hyponyms under 'Flower'.

6. Meronymy (Part-Whole Relationship)

Meronymy describes the PART-OF relationship. A meronym is a term for a part of something.

- 'Wheel' is a meronym of 'Car' (wheel is part of car).
- 'Finger' is a meronym of 'Hand'.
- 'Page' is a meronym of 'Book'.

7. Semantic Fields / Semantic Field Theory

Words can be grouped into semantic fields — sets of words related by a shared topic or domain. E.g., the semantic field of 'cooking': boil, fry, sauté, bake, roast, steam, grill.

Relation	Definition	Example Pair	NLP Use
Homonymy	Same form, unrelated meanings	bank (river/finance)	WSD
Polysemy	Same form, related meanings	head (body/leader)	WSD
Synonymy	Same/similar meaning	big / large	Query expansion
Antonymy	Opposite meaning	hot / cold	Sentiment, contrast
Hyponymy	Is-a (specific/general)	poodle / dog	Knowledge graphs
Meronymy	Part-whole	wheel / car	Ontology building

Q9. Probabilistic Context-Free Grammar (PCFG)

Context-Free Grammar (CFG) Recap

A CFG is a formal grammar with rules of the form: $A \rightarrow \beta$, where A is a non-terminal symbol and β is a sequence of terminals and/or non-terminals. A sentence is grammatical if it can be derived from the start symbol using the rules.

Definition of PCFG

A Probabilistic Context-Free Grammar (PCFG), also called a Stochastic Context-Free Grammar (SCFG), is a CFG where each production rule is assigned a probability. These probabilities represent how likely each rule is to be used when expanding a non-terminal symbol.

Formally, a PCFG is a 5-tuple: $G = (V, T, R, S, P)$ where:

- V = set of non-terminal symbols (e.g., S, NP, VP)
- T = set of terminal symbols (words)
- R = set of production rules
- S = start symbol
- P = probability function: for each non-terminal A , $\sum P(A \rightarrow \beta) = 1$

Example PCFG

Rule	Probability	Meaning
$S \rightarrow NP VP$	1.0	Every sentence has NP and VP
$NP \rightarrow Det N$	0.7	70% of NPs are Det+N
$NP \rightarrow NP PP$	0.2	20% of NPs have a PP
$NP \rightarrow Pronoun$	0.1	10% of NPs are pronouns
$VP \rightarrow V NP$	0.6	60% of VPs are transitive
$VP \rightarrow V$	0.4	40% of VPs are intransitive
$Det \rightarrow 'the'$	0.6	
$Det \rightarrow 'a'$	0.4	

Probability of a Parse Tree

The probability of a parse tree T is the product of the probabilities of all rules used in the tree:

$$P(T) = \prod P(A \rightarrow \beta) \text{ for each rule used}$$

For the sentence 'the dog chased a cat':

$$\begin{aligned}
 P(\text{parse tree}) = & P(S \rightarrow NP VP) \times P(NP \rightarrow Det N) \times P(Det \rightarrow 'the') \times P(N \rightarrow 'dog') \\
 & \times P(VP \rightarrow V NP) \times P(V \rightarrow 'chased') \times P(NP \rightarrow Det N) \\
 & \times P(Det \rightarrow 'a') \times P(N \rightarrow 'cat')
 \end{aligned}$$

Benefits of Probabilistic Parsing

1. Disambiguation: When a sentence has multiple parse trees (structural ambiguity), PCFG selects the most probable one. The tree with highest $P(T)$ is chosen.
2. Ranking Parses: Can rank all possible parses by probability, not just find any one parse.
3. Robustness: Can handle grammatically incorrect or unusual sentences by assigning low (but non-zero) probabilities.
4. Learned from Data: Probabilities can be estimated from annotated treebanks (e.g., Penn Treebank) using Maximum Likelihood Estimation: $P(A \rightarrow \beta) = \text{Count}(A \rightarrow \beta) / \text{Count}(A)$.

- 5. Integration with Language Models: Can be combined with lexical probabilities for better performance.
- 6. Application in Speech Recognition: Helps disambiguate between acoustically similar sentences.

Limitations of PCFG

- Probabilities are rule-based and don't consider lexical dependencies (e.g., 'ate' prefers 'pizza' as object, not 'idea').
- Context-free assumption may be too strong for some linguistic phenomena.
- Lexicalized PCFGs (Collins Parser) address lexical dependency limitations.

Q10. Weather Prediction Using State Transition Matrix (HMM)

Hidden Markov Model (HMM) Overview

A Hidden Markov Model (HMM) is a statistical model where the system being modeled is assumed to be a Markov process with unobserved (hidden) states. In NLP, HMMs are used for POS tagging, speech recognition, etc.

An HMM has:

- States: Hidden states $S = \{s_1, s_2, \dots\}$ (e.g., weather: Sunny, Rainy, Cloudy)
- Observations: Observable outputs $O = \{o_1, o_2, \dots\}$ (e.g., activities: Walk, Shop, Clean)
- Transition Probabilities: $A = P(\text{next state} \mid \text{current state})$
- Emission Probabilities: $B = P(\text{observation} \mid \text{state})$
- Initial Probabilities: $\pi = P(\text{starting in each state})$

Weather Prediction Example

States (Hidden): Sunny (S), Rainy (R), Cloudy (C)

Observations (Visible): Walk (W), Shop (Sh), Clean (Cl)

State Transition Matrix A

From \ To	Sunny	Rainy	Cloudy
Sunny	0.6	0.2	0.2
Rainy	0.1	0.5	0.4
Cloudy	0.3	0.4	0.3

Emission Matrix B

State \ Observation	Walk	Shop	Clean
Sunny	0.7	0.2	0.1

Rainy	0.1	0.3	0.6
Cloudy	0.4	0.4	0.2

Initial Probabilities π

$$\pi(\text{Sunny}) = 0.5, \quad \pi(\text{Rainy}) = 0.3, \quad \pi(\text{Cloudy}) = 0.2$$

Problem: Given observations $O = [\text{Walk, Shop, Clean}]$, find most likely weather sequence.

Solution Using the Viterbi Algorithm

The Viterbi algorithm finds the most likely hidden state sequence given the observations. It uses dynamic programming.

Step 1: Initialize (t=1, Observation=Walk)

$$V1(\text{Sunny}) = \pi(S) \times B(S, \text{Walk}) = 0.5 \times 0.7 = 0.350$$

$$V1(\text{Rainy}) = \pi(R) \times B(R, \text{Walk}) = 0.3 \times 0.1 = 0.030$$

$$V1(\text{Cloudy}) = \pi(C) \times B(C, \text{Walk}) = 0.2 \times 0.4 = 0.080$$

Step 2: Recurse (t=2, Observation=Shop)

$$\begin{aligned} V2(\text{Sunny}) &= B(S, \text{Shop}) \times \max[V1(S) \times A(S, S), V1(R) \times A(R, S), V1(C) \times A(C, S)] \\ &= 0.2 \times \max[0.350 \times 0.6, 0.030 \times 0.1, 0.080 \times 0.3] \\ &= 0.2 \times \max[0.210, 0.003, 0.024] = 0.2 \times 0.210 = 0.0420 \end{aligned}$$

$$\begin{aligned} V2(\text{Rainy}) &= 0.3 \times \max[0.350 \times 0.2, 0.030 \times 0.5, 0.080 \times 0.4] \\ &= 0.3 \times \max[0.070, 0.015, 0.032] = 0.3 \times 0.070 = 0.0210 \end{aligned}$$

$$\begin{aligned} V2(\text{Cloudy}) &= 0.4 \times \max[0.350 \times 0.2, 0.030 \times 0.4, 0.080 \times 0.3] \\ &= 0.4 \times \max[0.070, 0.012, 0.024] = 0.4 \times 0.070 = 0.0280 \end{aligned}$$

Step 3: Recurse (t=3, Observation=Clean)

$$\begin{aligned} V3(\text{Sunny}) &= 0.1 \times \max[0.0420 \times 0.6, 0.0210 \times 0.1, 0.0280 \times 0.3] \\ &= 0.1 \times \max[0.0252, 0.0021, 0.0084] = 0.00252 \end{aligned}$$

$$\begin{aligned} V3(\text{Rainy}) &= 0.6 \times \max[0.0420 \times 0.2, 0.0210 \times 0.5, 0.0280 \times 0.4] \\ &= 0.6 \times \max[0.0084, 0.0105, 0.0112] = 0.6 \times 0.0112 = 0.00672 \end{aligned}$$

$$\begin{aligned} V3(\text{Cloudy}) &= 0.2 \times \max[0.0420 \times 0.2, 0.0210 \times 0.4, 0.0280 \times 0.3] \\ &= 0.2 \times \max[0.0084, 0.0084, 0.0084] = 0.00168 \end{aligned}$$

Result

At t=3: Rainy has maximum probability (0.00672).

Backtracking through the Viterbi path: The most likely weather sequence is:

t=1 (Walk) → Sunny (probability 0.350)

t=2 (Shop) → Sunny (probability 0.042)

t=3 (Clean) → Rainy (probability 0.00672)

Most likely sequence: [Sunny, Sunny, Rainy]

Q11. Morphology – Stems and Affix Classes of Morphemes

Definition of Morphology

Morphology is the branch of linguistics concerned with the internal structure and formation of words. It studies how words are built from morphemes — the smallest units of meaning or grammatical function in a language.

```
Word: 'unhappiness'
Morphemes: un- + happy + -ness
un-      = prefix (negative)
happy    = stem (root/base)
-ness    = suffix (converts adj to noun)
```

Types of Morphemes

1. Stem (Base / Root)

The stem (also called root or base) is the core morpheme that carries the primary lexical meaning of the word. All other morphemes are attached to it.

- 'teach' in teacher, teaching, teaches, taught
- 'happy' in unhappy, happiness, happily
- 'act' in actor, action, react, interaction
- 'break' in unbreakable, breakdown, breakable

A root is the most basic form (may not be a real word alone: 'ceive' in receive, conceive). A stem is the form to which affixes are added (may already have affixes: 'teacher' = stem for 'teachers').

Affix Classes

Affixes are bound morphemes that attach to a stem. They are classified by their position relative to the stem:

2. Prefixes

Added BEFORE the stem. Usually changes meaning, not POS.

Prefix	Meaning	Example
un-	not / reverse	happy → unhappy, do → undo
re-	again	write → rewrite, do → redo
pre-	before	view → preview, school → preschool
mis-	wrongly	understand → misunderstand
dis-	not / opposite	agree → disagree
anti-	against	social → antisocial
over-	too much	react → overreact

sub-	under/below	way → subway
------	-------------	--------------

3. Suffixes

Added AFTER the stem. Most commonly changes the POS (derivational) or grammatical features (inflectional).

Suffix	Function	Example
-ness	Adj → Noun	happy → happiness
-tion / -sion	Verb → Noun	act → action, decide → decision
-er / -or	Verb → Noun (agent)	teach → teacher
-ly	Adj → Adverb	quick → quickly
-ful	Noun → Adj	care → careful
-able / -ible	Verb → Adj	read → readable
-ize / -ise	Noun/Adj → Verb	modern → modernize
-ed	Verb (past tense, inflection)	walk → walked
-ing	Verb (participle, inflection)	run → running
-s / -es	Noun (plural, inflection)	cat → cats

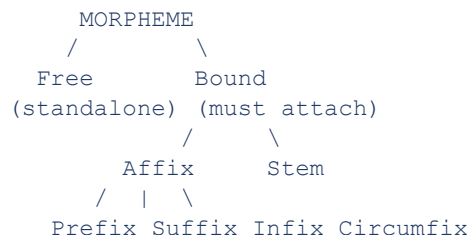
4. Infixes

Inserted WITHIN the stem. Rare in English, but common in other languages (Tagalog, Arabic). English informal example: 'abso-bloody-lutely', 'fan-freaking-tastic'.

5. Circumfixes

Wrap AROUND the stem — both a prefix and suffix added simultaneously. Example in German: ge- ... -t for past participle: 'machen' (to make) → 'gemacht' (made).

Summary Diagram



Q12. Context-Free Grammar (CFG)

Definition

A Context-Free Grammar (CFG) is a formal grammar in which each production rule has exactly one non-terminal symbol on the left-hand side. The rules can be applied regardless of the context (surrounding symbols) in which the non-terminal appears — hence 'context-free'.

Formally, a CFG is a 4-tuple: $G = (V, \Sigma, R, S)$ where:

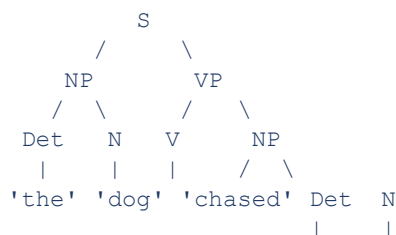
- V = finite set of non-terminal (variable) symbols, e.g., $\{S, NP, VP, PP, Det, N, V\}$
- Σ = finite set of terminal symbols (actual words), e.g., $\{the, dog, chased, cat\}$
- R = finite set of production rules of the form $A \rightarrow \beta$ (where $A \in V, \beta \in (V \cup \Sigma)^*$)
- S = start symbol (typically 'S' for Sentence)

Example CFG for English

Rule	Expansion	Meaning
$S \rightarrow NP VP$		Sentence = Noun Phrase + Verb Phrase
$NP \rightarrow Det N$		Noun Phrase = Determiner + Noun
$NP \rightarrow Det N PP$		Noun Phrase with prepositional phrase
$NP \rightarrow PRP$		Noun Phrase = Pronoun
$VP \rightarrow V$		Intransitive verb phrase
$VP \rightarrow V NP$		Transitive verb phrase
$VP \rightarrow VP PP$		Verb phrase with prepositional modifier
$PP \rightarrow P NP$		Prepositional phrase = Prep + NP
$Det \rightarrow 'the' \mid 'a'$		Determiners
$N \rightarrow 'dog' \mid 'cat' \mid 'park'$		Nouns
$V \rightarrow 'chased' \mid 'saw'$		Verbs
$P \rightarrow 'in' \mid 'with' \mid 'on'$		Prepositions

Parse Tree Example

Sentence: "The dog chased the cat"



'the' 'cat'

Chomsky Normal Form (CNF)

Any CFG can be converted to CNF, where all rules are either $A \rightarrow BC$ (two non-terminals) or $A \rightarrow a$ (one terminal). CNF is required for efficient parsing algorithms like CYK (Cocke-Younger-Kasami).

Original: $S \rightarrow NP VP PP$

CNF: $S \rightarrow NP X1, X1 \rightarrow VP PP$ (introduce new non-terminal $X1$)

Parsing Algorithms for CFG

- Recursive Descent (top-down, left-to-right): Simple but may loop infinitely with left-recursive rules.
- Earley Parser: Handles all CFGs including ambiguous ones; $O(n^3)$ time complexity.
- CYK (Cocke-Younger-Kasami): Requires CNF; $O(n^3)$ dynamic programming algorithm.
- LALR/LR Parsers: Used in compiler construction; handle deterministic CFGs efficiently.

Limitations of CFG

- Cannot model long-distance dependencies in some languages.
- Does not capture lexical preferences (e.g., 'eat' prefers 'food' over 'idea' as object).
- Cannot distinguish between subject and object NPs based on semantics.
- Extension: PCFG adds probabilities; Lexicalized CFG adds head-word information.

Q13. Techniques for Semantic Analysis

What is Semantic Analysis?

Semantic analysis is the process of understanding the meaning of natural language at the word, phrase, sentence, and discourse level. It goes beyond syntactic structure to determine WHAT a sentence means.

1. Lexical Semantic Analysis

Analyzes word-level meaning. Key techniques:

- Word Sense Disambiguation (WSD): Determining which sense of a polysemous/homonymous word is intended. 'I went to the bank' — financial institution or river bank? Uses context windows, Lesk algorithm, or neural models.
- Named Entity Recognition (NER): Identifies and classifies named entities: Person (John Smith), Location (Paris), Organization (Google), Date (March 2026). Uses CRF, BiLSTM-CRF, BERT-based models.
- Semantic Similarity: Measuring how similar two words/phrases are in meaning using vector space models (Word2Vec, GloVe) or knowledge bases (WordNet).

2. Compositional Semantics (Formal Semantic Representation)

Builds the meaning of a sentence compositionally from its parts using formal logic.

- First-Order Logic (FOL): 'Every dog barks' $\rightarrow \forall x \text{ Dog}(x) \rightarrow \text{Barks}(x)$
- Lambda Calculus: Represents semantic composition formally using λ -notation.
- Semantic Parsing: Maps natural language to formal representations (SQL, logic forms) for question answering systems.

Sentence: "John ate an apple."

FOL: $\exists e \text{ Ate}(e) \wedge \text{Agent}(e, \text{John}) \wedge \text{Patient}(e, \text{apple})$

3. Semantic Role Labeling (SRL)

SRL identifies the semantic roles of constituents in a sentence relative to a predicate:

- Agent (ARG0): The doer of the action — 'John threw the ball.' $\rightarrow$ John = Agent
- Patient (ARG1): The entity acted upon — ball = Patient
- Instrument: 'She cut the bread with a knife.' $\rightarrow$ knife = Instrument
- Location, Time, Manner, Purpose, etc.

Frameworks: PropBank, FrameNet, VerbNet. Tools: AllenNLP SRL (based on BERT).

4. Coreference Resolution

Determines when different expressions in a text refer to the same real-world entity.

'Maria told Lisa that she had won the prize.'

$\rightarrow$ Who is 'she'? Coreference resolution determines: she = Maria or Lisa

Methods: Rule-based (Hobbs algorithm), Machine Learning (mention-pair models), Neural (end-to-end coreference, SpanBERT).

5. Distributional Semantics (Vector Space Models)

Based on the Distributional Hypothesis: 'Words that occur in similar contexts have similar meanings.' Represents words as dense vectors in high-dimensional space.

- Word2Vec (CBOW, Skip-gram): Neural network-based word embeddings.
- GloVe (Global Vectors): Based on word co-occurrence matrices.
- FastText: Subword-level embeddings; handles OOV words.
- BERT, GPT: Contextual embeddings — same word has different vectors in different contexts.

6. Sentiment Analysis / Opinion Mining

Determines the sentiment (positive, negative, neutral) expressed in a piece of text. Techniques: Lexicon-based (SentiWordNet), ML classifiers (SVM, Naive Bayes), Deep learning (LSTM, BERT fine-tuned).

Technique	Input	Output	Tool/Model
WSD	Ambiguous word in context	Correct sense	Lesk, BERT
NER	Text with entities	Entity labels	spaCy, BERT-NER

SRL	Sentence with predicate	Role labels	AllenNLP
Coreference	Document with pronouns	Entity clusters	SpanBERT
Sentiment	Review/text	Pos/Neg/Neutral	VADER, BERT

Q14. Text Processing in Natural Language Processing

Overview

Text processing refers to the series of steps applied to raw text data to clean, normalize, and transform it into a form suitable for NLP tasks. It forms the pre-processing pipeline before any ML/DL model or NLP algorithm can be applied.

Text Processing Pipeline
1. Text Acquisition → Collecting raw text data
2. Text Cleaning → Removing HTML, special characters, noise
3. Sentence Segmentation → Splitting into sentences
4. Tokenization → Splitting into words/subwords
5. Case Normalization → Converting to lowercase
6. Stop-word Removal → Removing function words
7. Stemming / Lemmatization → Reducing to root form
8. POS Tagging → Labeling grammatical roles
9. Named Entity Recognition → Identifying entities
10. Feature Extraction → Converting to numerical vectors (TF-IDF, embeddings)

1. Text Acquisition

Gathering raw text from sources: web scraping (BeautifulSoup, Scrapy), APIs (Twitter, Reddit), databases, PDFs, OCR on images, speech-to-text. Quality and diversity of data greatly affect downstream model performance.

2. Text Cleaning

Removing noise and irrelevant content:

- Removing HTML tags: `<p>Hello</p>` → Hello
- Removing URLs: `https://example.com` → [removed]
- Removing special characters and numbers (unless needed)
- Fixing encoding issues (UTF-8, ASCII)
- Handling emojis and Unicode characters

3. Case Normalization

Converting all text to lowercase to ensure 'Apple' and 'apple' are treated as the same token. However, case may be preserved for NER (proper nouns vs. common words).

'Hello World' → 'hello world'

4. Stop-word Removal

Removing high-frequency, low-information words (the, is, at, which, in, a, an, etc.) to reduce noise and dimensionality. Useful for text classification, IR; but harmful for tasks like NER, MT, QA.

'the cat sat on the mat' → 'cat sat mat'

5. Stemming

Reducing words to their approximate root/stem by stripping suffixes using heuristic rules. May produce non-words. Porter Stemmer, Snowball Stemmer.

'running', 'runner', 'runs' → 'run'

'studies', 'studying' → 'studi' (not a real word)

6. Lemmatization

Reducing words to their canonical dictionary form (lemma) using linguistic knowledge (vocabulary + morphological rules). Always produces valid words.

'running' → 'run' (verb), 'better' → 'good' (adj)

'geese' → 'goose' (noun)

7. Part-of-Speech Tagging

Assigning grammatical labels to each token. Used as input to higher-level tasks like parsing, NER, semantic analysis.

8. Vectorization / Feature Extraction

Converting text to numerical representations:

- Bag of Words (BoW): Frequency-based word counts.
- TF-IDF: Term Frequency × Inverse Document Frequency — down-weights common words.
- Word Embeddings: Word2Vec, GloVe — dense semantic vectors.
- Contextual Embeddings: BERT, GPT — context-dependent dense vectors.

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log(N / \text{DF}(t))$$

Q15. Stemming vs. Lemmatization

Introduction

Both stemming and lemmatization are text normalization techniques used to reduce word variants to a common base form. They reduce vocabulary size and help NLP models generalize. However, they differ significantly in their approach, accuracy, and linguistic soundness.

Feature	Stemming	Lemmatization
Definition	Heuristically strips suffixes/prefixes to get approximate root (stem)	Maps word to its canonical dictionary form (lemma) using linguistic knowledge

Output	May produce non-words (pseudo-roots)	Always a valid dictionary word
Accuracy	Lower; uses simple rules	Higher; uses morphological analysis
Speed	Faster (rule-based)	Slower (requires dictionary lookup)
Requires POS?	No	Yes — POS affects the lemma
Language knowledge	Minimal (just suffix rules)	Full morphological analysis
Example: 'running'	'run' OR 'runn'	'run'
Example: 'better'	'better' (unchanged)	'good' (adjective lemma)
Example: 'studies'	'studi' (not a word!)	'study' (correct!)
Example: 'geese'	'gees' (wrong)	'goose' (correct)
Common Tools	Porter, Snowball, Lancaster Stemmer (NLTK)	WordNet Lemmatizer (NLTK), spaCy, Stanford NLP

Examples Side by Side

Original Word	Stemmer Output	Lemmatizer Output
running	run	run
ran	ran	run
better	better	good
studies	studi	study
geese	gees	goose
caring	car	care
happily	happili	happily
wolves	wolv	wolf

When to Use Which?

- Use Stemming when: Speed is critical, vocabulary reduction is the primary goal, result quality is less important. Applications: Information Retrieval (search engines), simple text classification.
- Use Lemmatization when: Linguistic accuracy matters, the application requires real words, POS information is available. Applications: Machine translation, question answering, chatbots, semantic analysis.

Algorithms

Porter Stemmer (Most Popular Stemmer)

A series of 5 phases of rule-based suffix stripping. Example rules:

```
SSES → SS    (processes → process)
IES  → I      (ponies → poni)
ATIONAL → ATE (relational → relate)
```

WordNet Lemmatizer

Looks up words in the WordNet lexical database using POS to find the correct lemma. Handles irregular forms (be/was/were → be, good/better/best → good).

Q16. Parse Trees for "Astronomers saw the stars with ears" & Ambiguity

Understanding Syntactic Ambiguity

A sentence is syntactically ambiguous when it can be parsed in two or more different ways using the grammar rules, producing different parse trees — and importantly, different meanings.

The sentence 'Astronomers saw the stars with ears' is a classic example of PP-Attachment Ambiguity: the prepositional phrase 'with ears' can attach to either the Verb Phrase (VP) or the Noun Phrase (NP 'the stars'), giving two different interpretations.

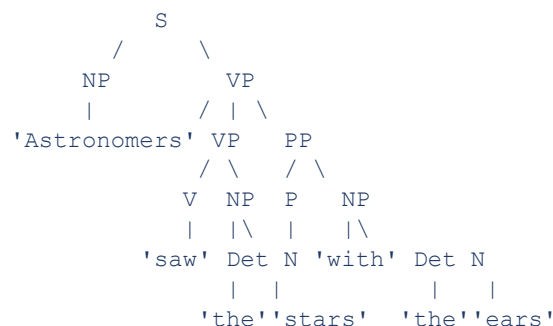
Grammar Rules Used

```
S    → NP VP
NP   → Det N | NP PP
VP   → V NP | VP PP
PP   → P NP
Det  → 'the'
N    → 'Astronomers' | 'stars' | 'ears'
V    → 'saw'
P    → 'with'
```

Parse Tree 1: PP Attached to VP

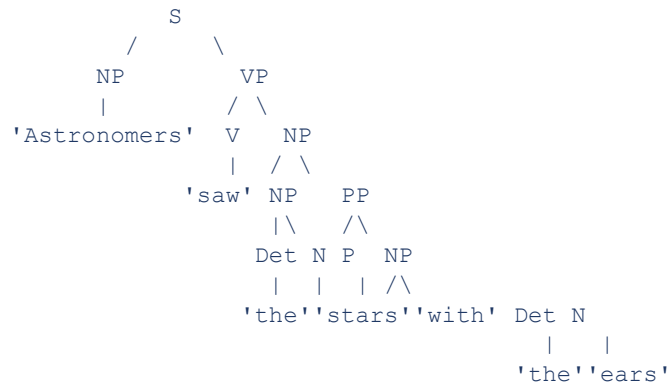
Interpretation: 'Astronomers used their ears to see the stars.'

(The PP 'with ears' modifies the VP 'saw the stars' — it is an instrument)



Parse Tree 2: PP Attached to NP

Interpretation: 'Astronomers saw the stars that have ears.'
(The PP 'with ears' modifies the NP 'the stars' — it is a modifier of stars)



Two Interpretations Compared

	Parse Tree 1 (PP → VP)	Parse Tree 2 (PP → NP)
PP attaches to	VP (verb phrase)	NP (noun phrase 'stars')
Meaning	Astronomers used ears to see	Stars that have ears were seen
Instrument/Modifier	PP = instrument of seeing	PP = attribute of stars
More plausible?	Biologically possible (if odd)	Stars don't have ears (very odd)

Types of Ambiguity Illustrated

- PP-Attachment Ambiguity: The prepositional phrase can attach to different constituents.
- Structural Ambiguity: Multiple parse trees correspond to different sentence structures.
- Semantic Ambiguity: Different structures give different meanings.

Resolving Ambiguity

- PCFG: Assign probabilities to rules; choose the highest-probability parse.
- Lexical preferences: 'saw ... with ears' — 'saw' doesn't commonly go with 'ears' as instrument.
- World knowledge: Stars don't have ears — pragmatic reasoning eliminates Parse Tree 2.
- Neural Parsing: Modern models (BERT-based parsers) use contextual embeddings to resolve attachment.

Q17. Finite State Transducer (FST) for Morphological Analysis

What is a Finite State Transducer (FST)?

A Finite State Transducer (FST) is a type of finite automaton that maps an input string to an output string. Unlike a regular Finite State Automaton (FSA) which only accepts or rejects strings, an FST produces an output for each input, making it perfect for morphological analysis and generation.

Formally, an FST is a 6-tuple: $T = (Q, \Sigma, \Delta, \delta, q_0, F)$ where:

- Q = finite set of states
- Σ = input alphabet
- Δ = output alphabet
- $\delta: Q \times \Sigma \rightarrow Q \times \Delta^*$ = transition function (maps state + input symbol $\rightarrow$ new state + output symbol)
- q_0 = initial state
- F = set of final/accepting states

FST vs FSA

Feature	FSA (Finite State Automaton)	FST (Finite State Transducer)
Purpose	Accept or reject strings	Map input strings to output strings
Output	Yes/No (accept/reject)	Output string (transformation)
Use in NLP	Recognize valid words	Morphological analysis/generation
Tape	Single tape (input only)	Two tapes (input + output)

Morphological Analysis with FST

Morphological analysis is the process of decomposing a word surface form into its morphological components (lexeme + features).

Input (surface form): 'cats'
Output (analysis): 'cat+N+PL' (Noun, Plural)

Input: 'walked'
Output: 'walk+V+PAST' (Verb, Past Tense)

Input: 'running'
Output: 'run+V+PROG' (Verb, Progressive)

FST for Noun Plural: cat $\rightarrow$ cats, fox $\rightarrow$ foxes

The FST handles orthographic rules: words ending in 's', 'x', 'z', 'ch', 'sh' get '-es', others get '-s'.

States and Transitions

States: q_0 (start), q_1 (reading), q_{end} (final)

Transition Rules for Pluralization:

```

q0 → q1: read any consonant/vowel, output same character
q1 → q1: continue reading word characters
At end of word:
  If last char ∈ {s, x, z, ch, sh}:
    Output: +es  (fox → foxes)
  Else:
    Output: +s   (cat → cats)

```

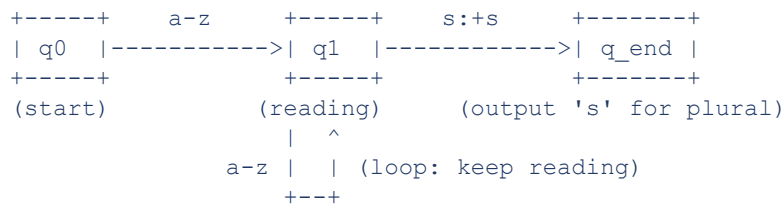
Detailed FST Example: Verb Morphology

FST for English verb morphology (past tense '-ed' and present progressive '-ing'):

Surface Form → Lexical Form Mapping

Input (Surface)	FST Output (Lexical)	Morphological Analysis
walked	walk + V + PAST	Verb, past tense (regular)
running	run + V + PROG	Verb, progressive (double 'n' rule)
danced	dance + V + PAST	Verb, past tense (drop 'e' + ed)
sitting	sit + V + PROG	Verb, progressive (double 't' + ing)
liked	like + V + PAST	Verb, past tense (drop 'e' + ed)
talking	talk + V + PROG	Verb, progressive (add 'ing')

FST Diagram for Simple Plural Formation



```

Special rule at q1 if last char = 'x':
x:x then ε:es → q_end
(fox → fox+es = foxes)

```

Bidirectionality of FSTs

A key advantage of FSTs is that they can be run in BOTH directions:

- Forward (Analysis): surface form → lexical form | 'cats' → 'cat+N+PL'
- Backward (Generation): lexical form → surface form | 'cat+N+PL' → 'cats'
- This bidirectionality makes FSTs elegant for both morphological analysis AND generation.

Composition of FSTs

Multiple FSTs can be composed (cascaded) to handle complex morphological phenomena:

```

FST1: Handle lexicon (stem lookup)

```

FST2: Handle morphological rules (affix attachment)

FST3: Handle orthographic/phonological rules (spelling changes)

Composed: FST1 ◦ FST2 ◦ FST3 → complete morphological analyzer

- Example system: XFST (Xerox Finite State Tools), FOMA — used for Arabic, Finnish, Turkish morphology.

Advantages of FST for Morphology

- Efficient: $O(n)$ time complexity for analysis and generation.
- Bidirectional: Same FST handles both analysis and generation.
- Compact: Represents large lexicons compactly using shared structure.
- Linguistically well-motivated: Maps directly to morphological theory.
- Composable: Multiple transducers can be composed into one.